Contents

Endpoint Inventory **1**
Conventions . . . 1
1. Identity — `/api/v1/auth` . . . 3
2. Profile and social graph — `/api/v1/me`, `/api/v1/users` . . . 7
3. Catalog — `/api/v1/tracks` . . . 9
4. Track Vote — `/api/v1/events` . . . 9
5. Playlist Editor — `/api/v1/playlists` . . . 11
6. Control Delegation — `/api/v1/devices` . . . 12
7. Playback — `/api/v1/playback-sessions` . . . 13
8. Audit and operations . . . 14
9. Realtime — client-to-server hub methods . . . 15
10. What is deliberately absent . . . 17
11. Open items . . . 17
Related . . . 17

Endpoint Inventory

The API contract, written before the code. This is [[delivery-plan]] Phase 3, and it is where the two workstreams meet and nowhere else. The client is built after the backend, so this file is read by the backend first and by the UI developer later; it must be correct for both.

Status: design, nothing implemented. Once code exists, `docs/api/openapi.json` generated from that code becomes the machine-readable contract and this file becomes the human overview. They must not diverge; a change lands in both in the same pull request.

Rules this document obeys: [[rest-api-design]] · [[openapi-documentation]] · [[signalr-realtime]] · [[authentication-authorization]] · [[owasp-api-top10-checklist]]

Conventions

Aspect	Value
Base path	`/api/v1`
Format	JSON, `camelCase`, UTF-8, UTC ISO-8601 instants
Errors	RFC 9457 `application/problem+json`, one shape, always with `traceId`

Aspect	Value
Validation error keys	**The field name the caller sent**, in `camelCase`, matching the request schema in `openapi.json`, and **never empty** — [[ADR-0043-validation-error-key-naming]], SPEC-037. A `422` body is `{"errors":{"<field>":["<message>"]}}`, so a client can take a key and look it up in the body or query string it just sent: `newPassword`, `youtubeVideoId`, `q` on `GET /tracks`, `ttlSeconds` on the event-code route. A nested field keeps its dotted path (`bio.value`, `geofence.latitude`); an entry in an array carries its index (`genres[0]`). One resolver produces this for every validator, so it is a property of the API and not a habit per route — `ValidationKeyContractTests` asserts every key against this document
Validation error codes	**A stable code beside every message** — [[ADR-0045-stable-error-codes]], SPEC-038. A `422` carries `codes` with the same keys as `errors` and one code per message, in the same order; a refusal that names no field — a wrong current password, for instance — carries a single `code` member instead. The text stays English and is for a developer; the code is what a client switches on and translates. A code is a contract: it is never reworded, recased or reused, and the vocabulary is `MusicRoom.Application.Validation.ValidationCodes`. Both members are declared in `openapi.json`, so a generated client sees them
Unexpected failures	**`500` with the same envelope** — [[ADR-0045-stable-error-codes]], SPEC-039, issue O29. A failure nobody decided answers `ProblemDetails` carrying `code: INTERNAL.UNEXPECTED` and a fixed title, and **nothing about itself**: no message, no exception type, no stack trace, no identifier. The detail is in an `Error` log line with the trace identifier, where an operator reads it and a caller cannot. One code for every exception on purpose — a code per type would tell a caller which component failed
Collections	`{ "items": [...], "page": 1, "pageSize": 20, "total": 137 }`, `pageSize` capped at 100
Ids in JSON	strings, always
Auth	`Authorization: Bearer <access token>`
Mutation headers	`X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version` — **required and enforced** since SPEC-008: middleware refuses any `POST`, `PUT`, `PATCH` or `DELETE` that omits one, or whose value exceeds 64 characters, with a bodiless `400`. Reads and the health probes are unaffected. Each accepted request leaves one audit record carrying these three values (SDD §13.4)

Aspect	Value
Optimistic concurrency	`ETag` on the resource, `If-Match` on the mutation, mismatch means `412`

Authorisation notation

Two gates, both evaluated server-side on every call ([[sdd]] §13.2):

Symbol	Gate 1 — identity	Gate 2 — visibility and licence
`anon`	none	none
`auth`	any authenticated user	none
`self`	the caller owns the resource	none
`owner`	the caller owns the aggregate	none
`member`	the caller is an attendee or collaborator	visibility resolved first
`licence`	the caller passes the aggregate's `LicenceMode` check	includes presence proof where required

A resource the caller may not see returns `404`, never `403`. `403` would confirm it exists.

Status codes used

`200` ok · `201` created (+ `Location`) · `202` accepted (generic, non-enumerating) · `204` no content · `400` malformed or missing client headers · `401` unauthenticated · `403` authenticated but not permitted · `404` absent **or hidden from this caller** · `409` conflict · `412` precondition failed (stale `If-Match`) · `422` semantically invalid · `429` rate limited

---

1. Identity — `/api/v1/auth`

Subject V.1, V.5. Rules: [[authentication-authorization]], [[social-oauth]].

Method	Path	Body	Success	Failures	Auth
POST	`/auth/registrations`	`email`, `password`, `displayName`	202	400, 422, 429	`anon`
POST	`/auth/email-verifications`	`token`, or `email` + `code`	204	400, 410 expired, 429	`anon`
POST	`/auth/verification-emails`	`email`	202	429	`anon`
POST	`/auth/sessions`	`email`, `password`	200 token pair	401 generic, 429	`anon`
POST	`/auth/sessions/social`	`provider`, `code`, `codeVerifier`, `redirectUri`	200 token pair	401, 422 (`PROVIDER.UNKNOWN`, `SOCIAL.EMAIL_UNVERIFIED`, `SOCIAL.ADDRESS_HAS_ACCOUNT`), 429	`anon`
POST	`/auth/tokens`	`refreshToken`	200 rotated pair	401, 429	`anon`
DELETE	`/auth/sessions/current`	—	204	401	`auth`
GET	`/auth/sessions`	—	200 list	401	`self`
DELETE	`/auth/sessions/{sessionId}`	—	204	401, 404	`self`

Method	Path	Body	Success	Failures	Auth
POST	`/auth/password-reset-requests`	`email`	202	422, 429	anon
POST	`/auth/password-resets`	`newPassword`, and **either** `token` **or** `email` + `code`	204	400, 410, 422, 429	anon
POST	`/me/password`	`currentPassword`, `newPassword`	204	401, 422, 429	auth
GET	`/auth/social-links`	—	200 list	401	self
POST	`/auth/social-links`	`provider`, `code`, `codeVerifier`, `redirectUri`	201	409 already linked, 422	auth
DELETE	`/auth/social-links/{provider}`	—	204	409 last sign-in method	self

OpenID Provider surface — S8 The API becomes an OpenID Provider at S8 – [[ADR-0031-musicroom-as-openid-provider]]. Paths and parameter names below follow OpenID Connect Core 1.0 and are **not** ours to rename.

Method	Path	Request	Success	Errors	Auth
GET	`/.well-known/openid-configuration`	—	200 discovery document	—	anon
GET	`/.well-known/jwks.json`	—	200 public key set	—	anon
GET	`/oauth2/authorize`	`response_type=code`, `client_id`, `redirect_uri`, `scope`, `state`, `nonce`, `code_challenge`, `code_challenge_method=S256`	302 to `redirect_uri` with `code`	400 unknown client or unregistered `redirect_uri`, 302 with `error`, 401	auth
POST	`/oauth2/token`	form-encoded: `grant_type=authorization_code` + `code` + `code_verifier` + `client_id` + `redirect_uri`, or `grant_type=refresh_token` + `refresh_token`	200 access token, rotated refresh token, and an ID token on the code grant	400 `invalid_grant` or `unsupported_grant_type`, 401 `invalid_client`, 429	anon
GET	`/oauth2/userinfo`	—	200 claims the granted scopes allow	401	auth
POST	`/oauth2/revoke`	form-encoded `token`, optional `token_type_hint`	200, including for an unknown token	400 `unsupported_token_type` for `token_type_hint=access_token`, 429	anon

`code_challenge_method=S256` is required; `plain` is refused. There is no `grant_type=password`: OAuth 2.1

removes the resource-owner password-credentials grant and RFC 9700 deprecates it.

`GET /oauth2/authorize` takes a bearer access token — owner's answer A of 2026-10-11, SPEC-044. This API has no login page, no cookie session and no HTML, so a person who is not already signed in has nowhere to sign in: the client authenticates with the credential endpoints first and then drives the code flow. The consequence is stated rather than discovered — **this is a first-party provider**, recorded as an amendment in [[ADR-0031-musicroom-as-openid-provider]].

`POST /oauth2/token` is the one place where a refusal is not ProblemDetails. The request is form-encoded and the error body is `{"error":"invalid_grant"}`, both fixed by RFC 6749 §4.1 and §5.2, which a client's own OAuth library reads. ADR-0045's `code` member applies to refusals this API designed; these the specification already named. The authorization endpoint's two non-redirecting refusals *are* ProblemDetails, carrying `CLIENT.UNKNOWN` and `REDIRECT_URI.UNREGISTERED`.

An unregistered `redirect_uri` answers `400` and does not redirect. Sending an error to a URI nobody registered is the open redirect the registered list exists to prevent — RFC 6749 §4.1.2.1. Every later refusal *is* delivered to the registered URI, with `state` echoed.

UserInfo answers the claims the session's grant allows — `openid` gives `sub`, `email` gives `email` and `email_verified`, `profile` gives `name` (OpenID Connect Core 1.0 §5.4). A claim the grant does not allow is **omitted**, not returned as null. The scopes are read from the session the access token names, never from the request: a scope a caller could send is a scope a caller grants itself. **A session opened by `POST /auth/sessions` or `POST /auth/sessions/social` granted no scope and receives all three** — the caller is the account's own client, and `GET /me` already returns strictly more. A rotation inherits the grant, so refreshing never widens it.

Revocation answers `200` for a token it has never seen. RFC 7009 §2.2: a caller must not be able to tell a live token from a dead one, or the endpoint is an oracle for which tokens exist. Revoking a refresh token ends its **whole family**, because a session is a family — a revocation that left the rotated token alive would revoke nothing that matters. `token_type_hint=access_token` is the one refusal: a 15-minute signed access token cannot be revoked by this design, and `200` would tell a client its token is dead while it still opens every route.

The refresh grant answers no `id_token`. An ID token asserts an authentication event, and a refresh exchange is not one: nothing new was proven, a stored token was presented. OpenID Connect Core 1.0 §12.1 leaves it optional.

These join `POST /auth/sessions`, `POST /auth/sessions/social` and `POST /auth/tokens`; they do not replace them — owner's answer of 2026-10-11, SPEC-049. An earlier draft of this section said the opposite, and it was written before the authorization endpoint was given its auth column: under that answer the credential endpoints are the **precondition** of `GET /oauth2/authorize`, so removing them would leave the code flow with no way for anybody to be authenticated in the first place.

What each surface is for, so two ways in reads as a trade-off rather than indecision:

Surface	Who uses it	What it gives
`POST /auth/sessions`, `/auth/sessions/social`, `/auth/tokens`	this project's own client	the access and refresh tokens every other endpoint takes. One request, no redirect, no browser
`/oauth2/authorize` + `/oauth2/token`	a standard client, including ours when it wants an ID token	the authorization code flow with PKCE, an **ID token** a third party can verify against the JWKS, and the scope a UserInfo call is filtered by

Both answer the same session model: one refresh-token family, rotation on every use, reuse revokes the family (ADR-0032). Neither is a second credential path in the sense that matters — they are two front doors onto

one session, which is why the rate limits, the audit records and the revocation all apply identically.

A verification message carries a link and a six-character code — SPEC-052. They are two credentials on one record: the link's token is 256 bits, the code is six characters from a 32-symbol alphabet (`A-Z` without `I` and `O`, `2-9`), and spending either spends both. Send `token`, or send `email` with `code`; the address is required with a code because six characters are not unique across accounts.

A wrong code costs one of five attempts. The fifth spends the row, so the sixth attempt is refused even when the code is right, and the person asks for a new message through `/auth/verification-requests` — which is rate-limited per IP and per account. That budget, not the length, is what makes a short code safe.

Both last fifteen minutes, down from twenty-four hours: a credential typed out of an inbox should not sit in one overnight. A message delayed past the window needs a resend.

Case and spacing in a code do not matter. `abc-234` and `ABC234` are the same code; what a mail client puts on the clipboard works.

The reset message carries a code too — SPEC-055, which reverses SPEC-052's refusal of one. The same shape, the same alphabet, the same five-attempt budget, and the same rule that spending either credential spends the row: send `token`, or send `email` with `code`. Two differences from verification, both deliberate:

- **the window is one hour**, as it always was for a reset, and the code shares it — one row, one expiry, rather than a second window that eventually outlives the first;
- **the code path never answers `410`.** An expired token identifies its own row, so telling its holder it expired says nothing they did not know; a code is scoped by an address, and a `410` for one would say that address has a reset outstanding. By code, every refusal is `400`.

SPEC-052 refused a code here because six characters is a weak credential in front of the strongest action in the system. What makes it acceptable is what makes it acceptable for verification: five guesses out of 1.07e9 per issued row, and a sixth chance only through a rate-limited endpoint that mails the account's own address.

A wrong code, an unknown address, a spent row and an exhausted budget are one answer. A caller who could tell them apart could learn which addresses have a verification outstanding.

The password reset has no code — link only. A six-character credential that replaces a password is a weak credential for the strongest action in the system.

`Accept-Language` picks the language of every message, `tr` or English, with English as the default for anything else. It is a header rather than a stored preference, so the message a request causes is in the language that request was made in.

Design notes

- **`202` on registration, verification resend and password reset is deliberate.** A `409 email already exists` is an account-enumeration oracle. The response is identical whether or not the address is known; the mailbox tells the real story. Cost: a typo'd address gives no immediate feedback. Accepted, and stated at defence.
- `/auth/tokens` performs refresh **rotation**. Presenting an already-rotated refresh token revokes the whole family and returns `401` — this is the session-theft detector (T-03).
- `/auth/password-resets` revokes every refresh token for that account on success.
- **`POST /me/password` answers `422` for a wrong current password, not `401`** (SPEC-028). The caller's token is valid and the request is authenticated; what failed is a precondition, and a `401` would tell a well-behaved client its session had ended and send it into a refresh-then-retry loop over a typo. There is nothing to hide either — the caller already knows which account this is — so the message says which check failed.
- **The social routes take an authorization code, not an ID token** (SPEC-041, ADR-0008). The client runs the authorization step with PKCE and sends `code` and `codeVerifier`; **this API exchanges them** at the provider's token endpoint with the client secret, and validates the ID token that comes

back. An ID token posted by a client would be accepted on the strength of `aud` alone; a code the API exchanges itself is evidence the flow ran where this API started it.

- **A social sign-in creates an account the first time, and that account has no password.** Password sign-in for it answers the same generic `401` every wrong credential answers, and `POST /me/password` answers `422`: there is nothing to replace. Setting a first password is the reset flow, which already proves the address.
- **An address the provider has not verified is refused** with `422` and `SOCIAL.EMAIL_UNVERIFIED`. Matching an unverified address to an existing account is the account-takeover path ADR-0008 names.
- **An address that already has an account is refused** with `422` and `SOCIAL.ADDRESS_HAS_ACCOUNT` — SPEC-057. Same reason: the provider verified the address, but the account was not created through this identity, and linking on a matching address is the takeover path. **The way through is deliberate:** sign in with the password, then `POST /auth/social-links`, after which the provider signs that account in. The message says that and nothing about the account — no display name, no timestamp, no mention of how it signs in. *Why this one speaks when the others do not:* the provider asserted `email_verified` for the address, so the caller has already proved they control it. They learn nothing a password-reset request would not tell them. Every other refusal on the route stays a bodiless `401` — a refused code, a mismatched `redirect_uri`, a failed PKCE check — because in those the caller has proved nothing.
- **A password that was accepted yesterday can be refused today** (SPEC-035). Every route that chooses a password — `/auth/registrations`, `/auth/password-resets` and `/me/password` — now screens it against a set of known breached passwords bundled with the application and against the caller's own words: the project name, the address local part and the display name, matched by containment at four characters or more. The answer is the `422` those routes already give an unacceptable password, keyed `Password` on register and `NewPassword` on the other two, with a message that names the screen and never the password or the word that matched. **Signing in is not screened:** a password already on an account still works, so no client needs to handle a forced rotation. A refused reset does **not** spend the token, so the same link can be used again.
- **A change ends every *other* session; the caller's own survives**, because it is the session that just proved it holds the new credential. A reset cannot prove that and still revokes every family (ASVS V7.4.3). The route carries both halves of the credential limit, a per-IP policy and the per-account counter, so a stolen access token is not an oracle for guessing the password at request speed.
- **A session is a refresh-token family.** `GET /auth/sessions` returns one entry per live family with the `platform`, `device` and `appVersion` that started it, when it started and when it stops being renewable — never a token or a digest. `DELETE /auth/sessions/current` ends the family named by the access token's `sid` claim, so signing out of one device leaves the others signed in. `DELETE /auth/sessions/{sessionId}` ends a named one, and answers `404` for an id that is unknown, already ended or another account's.
- **An access token outlives a logout** by up to 15 minutes and 30 seconds, which is its remaining lifetime plus the accepted clock skew. Revoking the family stops renewal, not a token already signed — recorded as T-28.
- `/auth/sessions/social` both registers and signs in. Identity comes from the validated **ID token**, never an access token ([[ADR-0008-oidc-social-login]]).
- `DELETE /auth/social-links/{provider}` refuses with `409` when it would leave the account with no usable sign-in method.
- Every row here is rate limited per account **and** per IP.

---

2. Profile and social graph — `/api/v1/me`, `/api/v1/users`

Subject V.1. The friends graph exists because the `Friends` visibility tier requires it.

Method	Path	Body	Success	Failures	Auth
GET	`/me`	—	`200` full own profile	`401`	`self`

Method	Path	Body	Success	Failures	Auth
PATCH	`/me/profile`	any subset of profile fields, each with its `visibility`	`200`	`422`	`self`
PUT	`/me/music-preferences`	`genres[]`, `artists[]`	`200`	`422`	`self`
GET	`/users`	query: `q`, `page`, `pageSize`	`200` paged, public fields only	`401`	`auth`
GET	`/users/{userId}`	—	`200` **filtered per field**	`404`	`auth`
GET	`/me/friends`	paged	`200`	`401`	`self`
DELETE	`/me/friends/{userId}`	—	`204`	`404`	`self`
GET	`/me/friend-requests`	query: `direction=incoming|outgoing`	`200` paged	`401`	`self`
POST	`/me/friend-requests`	`targetUserId`	`201`	`409` duplicate or already friends, `404`	`auth`
PATCH	`/me/friend-requests/{requestId}`	`status`: `accepted | declined`	`200`	`404`, `409`	`self` (recipient)
DELETE	`/me/friend-requests/{requestId}`	—	`204`	`404`	`self` (sender)

Design notes

- `GET /users/{userId}` projects **per field** according to the requester's relationship (self / friend / stranger) against each field's `ProfileFieldVisibility`. The full row never leaves the server for the client to filter — OWASP API3. **Implemented in SPEC-013:** the decision is made in the SQL projection, so a field the reader may not see is never read at all; it is absent from the body, and a reader cannot tell a hidden field from an empty one. An account that does not exist answers the same sparse shape as one whose owner hid everything, so asking does not confirm existence — which is why this row has no `404`.
- **Search reads the display name only**, as a case-insensitive prefix through `ILIKE`, with `%` and `_` escaped so a query cannot become a wildcard probe. Matching a profile field would let a stranger confirm a friends-only value by searching for it. A blank query returns nothing rather than everybody, and `pageSize` is capped at 50.
- `PATCH /me/profile` sets both value and visibility in one call, so a new field cannot exist without a tier. A missing tier is `422`, never a silent default to public.
- `GET /users` search returns public fields only, paginated, rate limited. Unbounded user enumeration is OWASP API4.
- **The tiered profile fields are `realName`, `bio`, `location` and `birthDate`** (SPEC-011). Each is sent and returned as `{ "value": ..., "visibility": "Public" | "Friends" | "Private" }`, and a field sent without a `visibility` is `422` — the binder would otherwise default a missing enum to `Public`.
- **`displayName` is always public and carries no tier.** It is the handle an account is known by, and a search result or an attendee list has to show it. Do not build a visibility control for it.
- **Music preferences are private to their owner and carry no tier.** `PUT /me/music-preferences` replaces the whole set; both lists are bounded at 50 entries of 64 characters.
- **The friends graph is live** (SPEC-012), so `Friends` is now a tier that can be evaluated. A friendship is **symmetric and one row**: both parties see it, and there is no state in which one of them does not.
- `POST /me/friend-requests` answers `409` for three cases the caller can tell apart from what they attempted: already friends, a request already pending **in either direction**, and asking yourself. A

crossing request is not auto-accepted — nobody becomes a friend without accepting. An unknown account is `404`.
- Only the recipient may `PATCH` a request, and only the sender may `DELETE` one. A request that is not the caller's to act on answers `404`, as does one already answered.
- `DELETE /me/friends/{userId}` ends the friendship for both sides and clears the request rows between the pair, so they may befriend again later.

---

3. Catalog — `/api/v1/tracks`

Subject III. Rules: [[youtube-integration]].

Method	Path	Body	Success	Failures	Auth
GET	`/tracks`	query: `q`, `page`, `pageSize`	`200` paged `TrackRef`	`429`, `503` quota	`auth`
GET	`/tracks/{youtubeVideoId}`		`200` `TrackRef`	`404`, `503`	`auth`

Design notes

- `GET /tracks` is the **only** path that can reach `search.list`, the expensive YouTube call. It is debounced client-side, cached server-side, and rate limited. `GET /tracks/{id}` prefers the cache, then `videos.list`.
- The YouTube API key is server-side only. The client never calls YouTube for metadata.
- Quota exhaustion surfaces as `503` with a clear problem type, not a `500`.
- A `TrackRef` is `{ youtubeVideoId, title, channel, durationSeconds, thumbnailUrl }`. No media.

---

4. Track Vote — `/api/v1/events`

Subject V.2.1. The contended write path. Rules: [[realtime-architecture]], [[ADR-0014-location-time-licence]].

Method	Path	Body	Success	Failures	Auth
POST	`/events`	`name`, `visibility`, `licence`, `startsAtUtc`, `endsAtUtc`, `presenceProof?`, `geofence?`	`201`	`422`	`auth`
GET	`/events`	query: `scope=public|mine|invited`, paging	`200` paged	`401`	`auth`
GET	`/events/{eventId}`	—	`200`	`404` if hidden	`member`
PATCH	`/events/{eventId}`	`name`, `startsAtUtc`, `endsAtUtc`	`200`	`404`, `412`, `422`	`owner`
DELETE	`/events/{eventId}`	—	`204` soft delete	`404`	`owner`

Method	Path	Body	Success	Failures	Auth
PUT	`/events/{eventId}/licence`	`{ licence, licence, presenceProof?, geofence? }`	`200`	`404`, `422`	`owner`
POST	`/events/{eventId}/venue-codes`	`{ ttlSeconds }` (60–3600)	`201` short-lived code	`404`, `422`, `429`	`owner`
GET	`/events/{eventId}/invitations`	page params	`200`	`404`	`owner`
POST	`/events/{eventId}/invitations`	`{ userId }`	`201`	`404`, `409`	`owner`
DELETE	`/events/{eventId}/invitations/{userId}`		`204`	`404`	`owner`
POST	`/events/{eventId}/attendances`		`201`	`404`, `409`	`auth` if public, `member` if private
DELETE	`/events/{eventId}/attendances/current`		`204`	`404`	`self`
GET	`/events/{eventId}/queue`	page params, `pageSize`	`200` vote-ordered	`404`	`member`
POST	`/events/{eventId}/tracks`	`{ youtubeVideoId }`	`201` suggestion	`404`, `409` duplicate, `422`	`licence`
DELETE	`/events/{eventId}/tracks/{suggestionId}`		`204`	`404`	`owner` or suggester
PUT	`/events/{eventId}/tracks/{suggestionId}/vote`	`{ presence }`	`201`/`204`	`404`, `422`, `429`	`licence`
DELETE	`/events/{eventId}/tracks/{suggestionId}/vote`		`204`	`404`	`self`
PATCH	`/events/{eventId}/now-playing`	`{ suggestionId }`	`200`	`404`, `409`	`owner` or delegate

Design notes

- **The contended path is implemented** (SPEC-018): suggestions, votes, the vote-ordered queue, venue codes and the presence checks. Event `now-playing` waits for the `PlaybackSession` of slice S7, and `EventHub` is the third unit of S6.
- **The queue's order is counted from the vote rows on every read.** No column holds a vote count: a stored count is a second copy of a fact two writers can disagree about, and incrementing one is the lost update subject V.2.1 grades. Ties break on when the track was suggested, so the order is total and stable.
- **A failed presence proof is one `422` with one message.** Naming which part failed — an unknown code, an expired one, one already spent, or a coordinate too far away — would tell a caller whether a code exists and how close they got. A vote with no proof on a restricted event gets the same answer.
- **A venue code is single-use per account**, so one code read out to a room works once for each person in it. It lives 60 to 3600 seconds, is stored only as a SHA-256 digest, and issuing it is rate limited per account.
- **The aggregate, its licence policy, invitations and attendance are implemented** (SPEC-017). The contended write path — suggestions, votes, the queue, `now-playing` — and event codes are the second unit of slice S6; `EventHub` is the third.
- **The window is two required UTC instants and the end must be after the start.** Server time decides everything; a client clock is evidence of nothing ([[ADR-0014-location-time-licence]]).
- **A `LocationAndTimeRestricted` event must say which presence proof it requires**, and a `Geofence` or `Both` proof must carry a centre and a radius, or the answer is `422`. A licence that claims to restrict by place and time with nothing configured to check would read as a feature and behave as an open door. A presence policy on any other licence mode is `422` for the mirror-image reason: nothing would ever read it.
- **The `ETag` is the event's `xmin`**, as it is for a playlist, and `If-Match` is compared inside the `UPDATE`; an absent `If-Match` on the `PATCH` is `412`, not `428`.

- **Voting is `PUT`, not `POST`.** A vote is an idempotent fact: "this user votes for this track". A double submit from a flaky mobile network is then harmless — `201` the first time, `200` after. `POST` would create a second row and make the race worse.
- The vote's uniqueness is enforced by `ux_event_votes_event_id_suggestion_id_user_id` — a vote is for a *suggestion*, which is what the row names — and the duplicate is detected **by the database**, never by a read-then-write check ([[ef-core-persistence]]). A duplicate is `200`, not `409`: the fact the caller asked for is already true.
- The vote count is recomputed from the source rows inside the transaction. Nothing increments a counter held in memory.
- `presence` carries `{ coordinates?, eventCode? }` and is required only when the licence is `LocationAndTimeRestricted`. The server checks it against the event's geofence and window, using **server time**. The coordinate is used and discarded — it is never written to the vote row (threat T-21) and never logged.
- `POST /events/{eventId}/event-codes` is owner-only and rate limited; codes are short-lived and single-use per user.
- The broadcast on `EventHub` happens **after** commit, never before.
- **Attendance is not a gate on a public event.** Subject V.2.1 says that when an event is public, *all* users can find it and vote. `POST /attendances` records presence for the roster; it is never required before `PUT .../vote` on a public event. On a private event, the invitation is the gate, not the attendance.

---

5. Playlist Editor — `/api/v1/playlists`

Subject V.2.3. The second contended write path.

Method	Path	Body	Success	Failures	Auth
POST	`/playlists`	`name`, `visibility`, `licence`	`201`	`422`	`auth`
GET	`/playlists`	query: `scope=public|mine|shared`, paging	`200` paged	`401`	`auth`
GET	`/playlists/{playlistId}`	—	`200` + `ETag`	`404` if hidden	`member`
PATCH	`/playlists/{playlistId}`	`If-Match`, subset	`200`	`404`, `412`	`licence`
DELETE	`/playlists/{playlistId}`	—	`204` soft delete	`404`	`owner`
PUT	`/playlists/{playlistId}/visibility`	`visibility`, `licence`, `mode`	`200`	`422`	`owner`
GET	`/playlists/{playlistId}/collaborators`	paging	`200`	`404`	`member`
POST	`/playlists/{playlistId}/collaborators`	`userId`, `canEdit`	`201`	`404`, `409`	`owner`
DELETE	`/playlists/{playlistId}/collaborators/{userId}`	—	`204`	`404`	`owner`
GET	`/playlists/{playlistId}/tracks`	`If-None-Match`, `pageSize`	`200` + `ETag`	`404`	`member`
POST	`/playlists/{playlistId}/tracks`	`youtubeVideoId`, `afterTrackId?`, `atHead?`	`201`	`404`, `409` full, `412`, `422`	`licence`
PATCH	`/playlists/{playlistId}/tracks/{playlistTrackId}`	`If-Match`, `afterTrackId` or `beforeTrackId`	`200`	`404`, `412`, `422`	`licence`
DELETE	`/playlists/{playlistId}/tracks/{playlistTrackId}`	`If-Match`	`204`	`404`, `412`	`licence`

Design notes

- **The `ETag` is the row's PostgreSQL `xmin`** (SPEC-014), so it changes whenever the row does and no code has to remember to bump a version. `If-Match` is compared **inside** the `UPDATE`, so a stale writer changes nothing; an absent `If-Match` is treated as a mismatch and answered `412`, because the contract lists `412` and not `428` and both have the same fix — reload.
- **A playlist's licence is `Everyone` or `InvitedOnly` only.** `LocationAndTimeRestricted` is `422`: subject V.2.1 attaches location-and-time to an event, and accepting it here would imply a check nothing performs.
- **`member` is the owner or anybody named on the playlist, plus any authenticated caller when it is public.** `licence` is the owner, a collaborator with `canEdit`, or any member when the licence is `Everyone`. `owner` is the owner: a collaborator who may edit the contents does not decide who may see them.
- **`DELETE` is a soft delete.** The row stays, stamped, and a global query filter hides it from every read, so the audit trail keeps pointing at something that exists.
- **Reorder is a `PATCH` with `If-Match`.** The client sends the `ETag` it last read; a stale one gets `412` and the client reloads. Without this, two simultaneous moves silently overwrite each other — the exact failure subject V.2.3 grades.
- Position is a sparse ordering key computed **between the neighbours named in the request**, so inserting between two tracks does not rewrite the list.
- **Where a track goes is said explicitly** (SPEC-015). On `POST`, no destination appends, `atHead: true` inserts before everything, and `afterTrackId` inserts after that track; `atHead` together with `afterTrackId` is `422`. On `PATCH`, **exactly one** of `afterTrackId` or `beforeTrackId` is required, and neither or both is `422` — on a `PATCH` an absent field and a null field are the same request, so "null means the head" would make an empty body a silent reorder. Moving to the head is `beforeTrackId` naming the current first track.
- **The collection's `ETag` is the playlist's `xmin`**, and every track write updates the playlist row in the same transaction. That is what gives a list a version it cannot hold itself, and it makes the guarded `UPDATE` the lock that serialises concurrent reorders: one `200`, the rest `412`.
- **A playlist holds at most 500 tracks**, and the 501st is `409` (SPEC-015). The same video twice is allowed: a playlist that plays a track twice is a playlist, not a conflict.
- Conflicts that survive a reload-and-reapply return `409` rather than guessing the user's intent.
- `PlaylistHub` broadcasts `TrackMoved` with the new version after commit.

---

6. Control Delegation — `/api/v1/devices`

Subject V.2.2. Licence is **per device**, bound to the account.

Method	Path	Body	Success	Failures	Auth
POST	`/devices`	none — from `X-Client-*` headers	`201`	`409` already registered	`auth`
GET	`/devices`	—	`200` own devices	`401`	`self`
DELETE	`/devices/{deviceId}`		`204`	`404`	`self`
GET	`/devices/{deviceId}/control`		`200` + `ETag` current controller	`404`	`self` or controller
PUT	`/devices/{deviceId}/control`	`grabberUserId`	`200`	`404`, `412`, `422` not a friend	`owner`
DELETE	`/devices/{deviceId}/control`		`204` released	`404`	`owner` or controller

Method	Path	Body	Success	Failures	Auth
GET	`/me/control-grants`	paged	`200` grants held by me	`401`	`self`

Design notes

- **§6 is implemented** (SPEC-020). Control is a **single row with a state machine and a concurrency token** ([[sdd]] §11.2): `controller_user_id` on the device plus its `xmin`, compared inside the `UPDATE`. Two simultaneous claims: one wins, the other gets `412`. The loser loses **visibly** — a silent overwrite is the bug, and twenty parallel claims are tested for exactly one winner.
- **Null means the owner controls it.** A revoke writes null rather than the owner's id, so there is one spelling of that state; `GET .../control` still answers with the owner, because that is the question asked.
- **A device is the same device across app updates.** `409` is keyed on `(owner, platform, model)` and deliberately not on the app version.
- `granteeUserId` must be a friend of the owner. Otherwise `422`.
- `DELETE .../control` is allowed to both the owner (revoke) and the current controller (release).
- `POST /devices` **takes no body**: `platform`, `model` and `appVersion` come from `X-Client-Platform`, `X-Client-Device` and `X-Client-App-Version`, which every mutation must send anyway. A body repeating them would be a second source that can disagree with the audit record written for the same request.

---

7. Playback — `/api/v1/playback-sessions`

Subject V.2.1 (a highly-voted track plays earlier) and V.2.2 (control is per device). Model and rationale: [[ADR-0017-playback-session]].

Method	Path	Body	Success	Failures	Auth
POST	`/playback-sessions`	`deviceId`, `sourceType`, `sourceId`	`201` + `ETag`	`404`, `409` device already has a session, `422`	`owner` of the device
GET	`/playback-sessions/{sessionId}`		`200` + `ETag`	`404`	`member` of the source
GET	`/events/{eventId}/playback-session`		`200` + `ETag`	`404`	`member` of the event
GET	`/devices/{deviceId}/playback-session`		`200` + `ETag`	`404`	`self` or controller
POST	`/playback-sessions/{sessionId}/advance`	`expectedNowPlayingId`	`200` now-playing	`404`, `409` queue empty, `412` stale	controller
PATCH	`/playback-sessions/{sessionId}`	`nowPlayingId` (explicit jump)	`200`	`404`, `412`	controller
DELETE	`/playback-sessions/{sessionId}`		`204` stop playback	`404`	`owner` of the device

Design notes

- **§7 is implemented** (SPEC-021). A session is opened by the device's **owner** on a source they may see; the **controller** — the owner, or the friend they delegated to — advances and jumps; the owner stops it.

- **`expectedNowPlayingId` makes advance idempotent.** A duplicate call from a flaky mobile network finds its expectation stale and gets `200` with the current state — it never skips a second track. Without it, a retry silently loses a song.
- **Two kinds of staleness, two answers.** A stale `expectedNowPlayingId` is the retry above and is success. A stale `If-Match` is `412`: somebody else moved the session first. Conflating them would lose one guarantee or the other — the token stops two controllers advancing at once, the expectation stops one controller's retry skipping a song.
- **An advance with nothing left ends the session and answers `409`**, and `NowPlayingChanged` carries a null now-playing so listeners stop rather than waiting.
- **The session holds no controller of its own.** Who may command it is read from the device (`controller_user_id ?? owner_id`), so a revoke takes effect on the delegate's next command — SPEC-020 put the grant in one place and SPEC-021 keeps it there.
- **A device holds at most one *live* session**, enforced by a unique index filtered on `ended_at_utc IS NULL`. A stopped session keeps its row.
- **A guest can ask the event itself what it is playing** — `GET /events/{eventId}/playback-session`, SPEC-058. Same rule as the session route: whoever may see the event may see what it is playing, and a caller who may not see it gets the same `404` as an event with nothing on. The route exists because a guest has the event id and not the session id, and without it they had to wait for the next `NowPlayingChanged` — so somebody arriving mid-track missed the rest of the track. The index is per **device**, so two devices can legitimately play one event; the newest live session answers.
- **`PlaybackSessionDetail` carries two clocks** — SPEC-058: `nowPlayingStartedAtUtc`, stamped by the server in the same statement that moves `now_playing_id`, and `serverNowUtc`, what the server's clock read while answering. A client seeking to the right second computes `serverNowUtc - nowPlayingStartedAtUtc` and corrects for its own clock with the second value rather than trusting it. `nowPlayingStartedAtUtc` is null when nothing is playing, and on sessions that predate the column — a client reading null plays from the beginning, which is what it did before. `NowPlayingChanged` carries the same stamp, so a listener and a late joiner compute from one number.
- The server picks the next item: for an `Event`, the highest vote count with ties broken by earliest suggestion; for a `Playlist`, the next ordering key. **The client reports that the track ended; it never chooses what plays next.**
- `controllerUserId` on the session is what `PUT /devices/{id}/control` changes. That is the concrete meaning of "delegating music control".
- `NowPlayingChanged` broadcasts on `EventHub` or `PlaylistHub` **after** commit.
- A device holds at most one active session.

8. Audit and operations

Method	Path	Success	Auth	Note
GET	`/me/activity`	`200` paged own audit records	`self`	optional; the **write** side is the graded requirement (V.6), the read side is convenience
GET	`/health`	`200`	`anon`	liveness, minimal body
GET	`/health/ready`	`200` / `503`	`anon`	readiness: one member per dependency — see below
GET	`/openapi/v1.json`	`200`	`anon`	the generated contract; interactive UI only in `Development`

Audit records are **written** by a pipeline behaviour on every command — that is subject V.6, and it is not optional. See [[logging-observability]].

The readiness body — SPEC-053 `GET /health/ready` answers `{"status": "...", "<dependency>": "...", ...}`: one member per registered dependency, carrying its state and **nothing else** — no host, no connection string, no exception text, no key or part of one.

Member	States	Meaning
`status`	`Healthy`, `Degraded`	the worst state any member reports
`database`	`Healthy`, `Unhealthy`	a connection was opened. `Unhealthy` is the only thing that makes this route answer `503`
`catalog`	`Healthy`, `Degraded`	**whether a provider key is configured on this host.** It is never called
`social`	`Healthy`, `Degraded`	**which social providers are configured**, by name — SPEC-056. The provider is never called, and no client id or secret appears here

`503` comes from `Unhealthy` alone. A `Degraded` member is reported and the route still answers `200`, because readiness is about whether this service can serve. Signing in, profiles, playlists, events, votes, devices and playback touch no music provider; only `GET /tracks` does. A probe that failed here would take the service out of rotation for a third party's outage, or for one feature.

`catalog` never calls YouTube. A `search.list` costs 100 of the 10,000 daily quota units ([[ADR-0023-youtube-quota-budget]]), so a probe that reached the provider would spend the budget it exists to watch — and on a platform that wakes a sleeping instance per request, at a rate nobody chose. `Degraded` therefore means exactly one thing: `YOUTUBE_API_KEY` is not set, so `GET /tracks` answers `503`. **`social` works the same way** and for the same reasons: it reports the provider rows configuration produced, never fetches a discovery document, and `Degraded` means the four `GOOGLE_*` variables are not all set, so `POST /auth/sessions/social` answers `422 PROVIDER.UNKNOWN`. Whether a configured key still works is what a real search finds out, and the host's log then says which cause applied — an absent key, a rejected key, the quota, or the provider itself.

---

9. Realtime — client-to-server hub methods

Server-to-client events are in [[sdd]] §9.2 and [[signalr-realtime]]. Hub methods are API endpoints with a different transport, and the same authorisation applies.

Hub	Method	Effect	Auth
`EventHub`	`JoinEvent(eventId)`	join group `event:{eventId}`	visibility + licence, **re-checked on reconnect**
`EventHub`	`LeaveEvent(eventId)`	leave the group	`auth`
`PlaylistHub`	`JoinPlaylist(playlistId)`	join group `playlist:{playlistId}`	visibility + licence, re-checked on reconnect
`PlaylistHub`	`LeavePlaylist(playlistId)`	leave the group	`auth`

Hub	Method	Effect	Auth
`ControlHub`	—	auto-joins `user:{userId}` on connect	`auth`

`PlaylistHub` is **implemented** at `/hubs/playlists` (SPEC-016) and `EventHub` at `/hubs/events` (SPEC-019) and `ControlHub` at `/hubs/control` (SPEC-022). All three hubs of SDD §9.2 exist.

Design notes

- **Mutations do not happen over the hub.** Every state change goes through the REST endpoints above, so one authorisation pipeline and one validation pipeline cover both transports. The hub carries joins, leaves and server-pushed facts.
- A client asking to join `playlist:42` proves nothing. Group membership is authorised server-side, every time. A reconnect is a new connection and therefore a new join, so the gate is re-asked by construction rather than by remembering to.
- **An `EventHub` payload carries the authoritative value of what changed, not a row version** ([[ADR-0039-queue-broadcast-payloads]]). `VoteChanged` carries the suggestion's new count, so a client that missed a message converges rather than drifting; `TrackSuggested` and `QueueReordered` carry an id, which is idempotent by itself. An event's `xmin` is already the `If-Match` of its metadata `PATCH`, so touching the event row on every vote would stop its owner renaming it during the evening it runs. Clients re-read the queue on connect and reconnect, which is the reconcile point a version would otherwise provide.
- **Every playlist payload carries the playlist's version** — the same `xmin` the REST `ETag` carries, so a client that receives a version it did not expect knows it missed a message and resynchronises over REST. The four events are `TrackAdded`, `TrackMoved`, `TrackRemoved` and `CollaboratorChanged`.
- **A duplicate vote broadcasts nothing**, and neither does a refused write: the count every listener holds is already right.
- **A broadcast is sent after the transaction commits**, by a pipeline behaviour outside it ([[ADR-0038-realtime-broadcast-posture]]). A refused write — a `412`, a `409`, a `404` — sends nothing.
- Authentication uses the same JWT, via the documented SignalR `access_token` query parameter on the WebSocket handshake.
- **`ControlHub` has no invocable method.** Its group is `user:{userId}`, taken from the validated token and joined automatically on connect, so a client cannot ask to listen to somebody else's control because it cannot ask at all. Nothing on it is audited: ADR-0019 audits hub **methods**, and the control changes themselves are already audited on the REST side as `device.grant-control` and `device.release-control`.
- **A control change tells both sides.** A grant sends `ControlGranted` to the grantee **and** the owner; a revoke or release sends `ControlRevoked` to both; a grant that replaces an existing grant additionally sends `ControlTakenOver` to the controller it displaced. The owner's other clients are as wrong as the grantee's until they hear — the one that made the call got the `200`, and a phone in another pocket did not.
- **Client context is captured at the handshake**, not per invocation — a hub call carries no HTTP headers. The three values go in the **query string** of the handshake (`platform`, `device`, `appVersion`); a header is accepted too, for a client that can set one. A handshake without all three is answered `400` by the same middleware that refuses a REST mutation without them, so neither transport can produce an action nobody can attribute.
- **The token goes in the `access_token` query parameter**, the documented SignalR mechanism, and it is read **only** under `/hubs`. No REST route gains a second way to present a token.
- **The handshake can be refused three ways, and each says something different** (SPEC-026, [[ADR-0030-hub-rate-limits-and-origin]]):
  - `400` — the three client values are missing, as above.
  - `403` — the request carries an `Origin` that is not in `CORS_ALLOWED_ORIGINS`. A request with **no**

`Origin` passes: that is the native client, which is not a browser and sends none. The response names neither the list nor the reason, because a response that described the policy would be a way to probe it.
  - `429` with `Retry-After` — the caller already holds the concurrent-connection ceiling: **5 per account, 20 per address**. A slot frees when a connection closes.
- **A hub method may be invoked 60 times a minute per connection.** The next one throws a `HubException` saying to slow down, and the refusal is audited `refused` exactly as a refused join is. The limit is per connection, and the connection ceiling above is what bounds how many of them one caller may hold.
- **Join and leave are audited** by a hub filter writing the same record as the REST behaviour ([[ADR-0019-hub-action-auditing]]). Connect and disconnect are not — mobile reconnect noise would bury deliberate actions.

---

10. What is deliberately absent

Not present	Why
Any endpoint returning another user's private data	subject V.6; there is no "admin view" in this project
Bulk or unpaginated list endpoints	OWASP API4; every list is capped
A `PUT /users/{id}` style full replace	invites mass assignment; profile updates are `PATCH` with explicit fields
Media streaming or proxying	playback is client-side YouTube; we store only `TrackRef`
Direct database or Supabase access from a client	subject V.3 makes the backend the sole guardian of truth
Mutation over SignalR	keeps one authorisation and validation path

---

11. Open items

#	Item	Resolved by
E1	Concrete request and response schemas per endpoint	the OpenAPI document, generated from code
E2	Exact `PlaybackSession` and control state names	the S7 slice ([[sdd]] §11.2)
E3	Whether `GET /me/activity` ships	drop it first if S9 is tight; the write side is what is graded
E4	Pagination cursor vs offset for the event queue	offset assumed; revisit if the queue grows large

---

Related

[[sdd]] §9 · [[delivery-plan]] · [[rest-api-design]] · [[signalr-realtime]] · [[owasp-api-top10-checklist]] · [[musicroom-backend]] · [[musicroom-mobile]]